Sistema de Reconocimiento Automático de Placas Vehiculares (ALPR) para el Control de Acceso en Porterías de Conjuntos Residenciales

Institución: Institución Educativa Consejo Municipal La Estrella **Grado:** Once **Área:** Tecnología e Informática — Proyecto de Feria de la Ciencia **Fecha:** Septiembre de 2026

> **Nota sobre el estado del proyecto:** este documento describe un proyecto que **se encuentra en fase de construcción y desarrollo**. No es un trabajo terminado: el sistema ya funciona y ya se ha probado varias veces, pero seguimos mejorándolo semana a semana. Por eso, en varias secciones (sobre todo en los resultados y las conclusiones) se aclara qué está confirmado, qué está en proceso y qué falta por hacer. Preferimos mostrar el trabajo real, con sus avances y sus tropiezos, en vez de simular que ya todo está resuelto.

Índice

1. Observación y delimitación del tema
2. Planteamiento del problema y pregunta de investigación
3. Justificación y objetivos
4. Marco teórico y antecedentes
5. Formulación de la hipótesis
6. Diseño metodológico
7. Recolección de datos y experimentación
8. Procesamiento y análisis de resultados
9. Conclusiones

---

1. Observación y delimitación del tema

En muchos conjuntos residenciales de nuestras ciudades, el ingreso de vehículos a la portería todavía se controla de forma completamente manual: un vigilante mira la placa del carro o de la moto, la compara mentalmente (o en un cuaderno, o en una lista impresa) con

los vehículos autorizados, y decide si deja pasar o no. Este proceso, aunque parece sencillo, tiene varios problemas que cualquiera que haya entrado a un conjunto residencial en hora pico ha vivido: filas de carros esperando, errores al leer una placa parecida a otra, falta de un registro automático de quién entró y a qué hora, y una carga de atención muy alta para una sola persona que debe hacer esto decenas o cientos de veces al día.

Ese fue el hecho observado que dio origen a este proyecto: la posibilidad de apoyar (no reemplazar) esa tarea humana con un sistema que lea automáticamente la placa de un vehículo a partir de una foto o un video, y que le diga al vigilante, en segundos, si ese vehículo está autorizado.

Para que el proyecto fuera manejable en el tiempo disponible (un periodo escolar, con recursos de un colegio, sin acceso a cámaras de seguridad profesionales ni a un conjunto residencial real donde instalar el sistema), delimitamos el tema así:

- **Qué:** un sistema de software que reconoce automáticamente placas vehiculares colombianas (de carro y de moto) a partir de fotos y video, y que consulta si esa placa pertenece a un residente o a un visitante autorizado.
- **Dónde:** pensado para la portería de un conjunto residencial; probado, mientras tanto, con fotografías reales de vehículos tomadas en la ciudad (varios municipios del área metropolitana de Medellín) y con una maqueta a escala planeada para las últimas semanas del proyecto.
- **Cuándo:** desarrollado durante nueve semanas, entre agosto y septiembre de 2026, con pruebas y mejoras continuas registradas día a día.
- **Con qué:** herramientas de inteligencia artificial y visión por computador de acceso libre (no de pago), ejecutadas en un computador portátil normal, sin tarjeta gráfica especializada, más una tarjeta gráfica gratuita prestada por Google Colab únicamente para la parte de entrenamiento del modelo.

---

2. Planteamiento del problema y pregunta de investigación

El control manual de acceso vehicular en porterías presenta varias limitaciones: depende completamente de la atención humana (que se cansa y se distrae), no deja un registro digital automático de los ingresos, es más lento en horas de alto tráfico, y no ofrece ninguna ayuda cuando la placa es difícil de leer a simple vista (de noche, con lluvia, o cuando el vehículo pasa rápido).

A partir de esa situación, formulamos la siguiente pregunta de investigación:

> **¿Es posible construir un sistema de visión artificial capaz de reconocer automáticamente placas vehiculares colombianas —tanto de carro como de moto— con una precisión suficientemente útil para apoyar el control de acceso en la portería de un conjunto residencial, y cuáles son los factores (distancia de la cámara, ángulo de la toma, iluminación, tipo de vehículo, y si la imagen es sintética o real) que más afectan esa precisión?**

Esta pregunta cumple las condiciones que debe tener una buena pregunta de investigación:

- **No se responde con sí o no:** exige medir un porcentaje de precisión y explicar de qué depende.
- **No se responde con una simple búsqueda en internet:** aunque existen sistemas comerciales de reconocimiento de placas (ALPR, por sus siglas en inglés), ninguna búsqueda le va a decir a nuestro grupo qué tan bien funciona *nuestro propio sistema*, construido con *nuestras propias herramientas* y probado con *nuestras propias fotografías*. Eso solo se puede saber construyéndolo y midiéndolo.
- **Es respondible con datos:** se puede medir con números concretos (porcentaje de placas leídas correctamente, tiempo de procesamiento, efecto de cada condición).

---

3. Justificación y objetivos

3.1 Justificación

Este proyecto vale la pena por varias razones:

Beneficio práctico. Un sistema de este tipo, si se perfecciona, podría ayudar a los vigilantes de un conjunto residencial a trabajar con menos carga y menos margen de error, y le daría a la administración un registro automático de quién entra y sale, útil tanto para la seguridad como para tareas cotidianas (por ejemplo, saber a qué apartamento entregar un domicilio o un paquete cuando llega un visitante).

Vacío que se busca llenar. Los sistemas comerciales de reconocimiento de placas existen, pero suelen ser costosos y cerrados (no se puede ver ni modificar cómo funcionan por dentro), lo que los hace poco accesibles para conjuntos residenciales pequeños o medianos, que son la mayoría en nuestro entorno. Este proyecto explora si es posible construir algo funcional usando únicamente herramientas de acceso libre y un computador común, sin inversión en equipos costosos.

Valor educativo. Para nosotros como estudiantes, este proyecto es una oportunidad real de aprender cómo funciona la inteligencia artificial "por dentro": no solo usarla como una caja negra, sino entender qué es entrenar un modelo, por qué a veces falla, cómo se mide si algo mejoró de verdad, y qué significa que un programa "aprenda" de ejemplos en vez de seguir instrucciones escritas a mano. Es, además, un ejercicio real de método científico aplicado a la tecnología: formulamos hipótesis, medimos, nos equivocamos, corregimos y volvemos a medir.

3.2 Objetivo general

Desarrollar un sistema de reconocimiento automático de placas vehiculares colombianas que apoye el control de acceso en la portería de un conjunto residencial, evaluando experimentalmente su precisión bajo distintas condiciones de captura.

3.3 Objetivos específicos

1. **Diseñar y entrenar un modelo de detección** capaz de encontrar la ubicación de una placa dentro de una foto o un fotograma de video, usando imágenes generadas por computador (datos sintéticos) como material de entrenamiento.
2. **Implementar un módulo de lectura de caracteres** que convierta la imagen de la placa ya localizada en un texto (por ejemplo, "ABC123"), junto con una capa de corrección que aproveche las reglas del formato colombiano de placas para arreglar errores típicos de lectura.
3. **Construir una base de datos y una interfaz de usuario** que permitan registrar apartamentos, residentes, vehículos autorizados y visitantes, y que muestren al vigilante si una placa reconocida está o no autorizada a ingresar.
4. **Medir experimentalmente la precisión del sistema** bajo distintas condiciones controladas (distancia, ángulo, iluminación, tipo de vehículo) y, adicionalmente, con fotografías reales tomadas en la calle, para identificar qué factores afectan más el resultado y qué tanta diferencia hay entre un ambiente de prueba controlado y el mundo real.

Cada uno de estos objetivos corresponde, más adelante, a una parte concreta de la metodología (semanas de trabajo) y a un conjunto de resultados medibles.

---

4. Marco teórico y antecedentes

4.1 ¿Qué es la inteligencia artificial y el aprendizaje automático?

La inteligencia artificial (IA) es el campo de la informática que busca que un computador realice tareas que normalmente requieren "inteligencia" humana: reconocer una cara, entender una frase, o —en nuestro caso— leer una placa en una foto.

Dentro de la IA existe una rama llamada **aprendizaje automático** (*machine learning*), que es la que usamos en este proyecto. La idea central es sencilla: en vez de que un programador escriba, uno por uno, todas las reglas para reconocer una placa ("si hay un rectángulo amarillo, y dentro hay letras negras, y mide tanto..."), se le muestran al computador **miles de ejemplos** (imágenes con la placa ya marcada) y un algoritmo ajusta automáticamente miles de parámetros internos hasta que el programa "aprende" el patrón por su cuenta. A ese proceso de mostrar ejemplos y ajustar parámetros se le llama **entrenar un modelo**.

Esto es distinto de la programación tradicional, donde uno escribe instrucciones exactas paso a paso. Aquí uno construye el "molde" (la arquitectura del modelo) y el proceso de entrenamiento, pero el conocimiento final —qué combinación de píxeles significa "aquí hay una placa"— lo descubre el propio algoritmo a partir de los datos, no lo escribe el programador a mano.

4.2 Visión por computador

La **visión por computador** es la rama de la inteligencia artificial dedicada a que un computador "interprete" imágenes o video. Dentro de esta rama, dos tareas nos interesan particularmente:

- **Detección de objetos:** encontrar *dónde* está algo dentro de una imagen (en nuestro caso, dónde está la placa dentro de la foto completa de un carro).
- **Reconocimiento óptico de caracteres (OCR):** convertir una imagen que contiene texto en el texto mismo, como cadena de caracteres que el computador puede procesar (convertir la imagen de "ABC123" en el texto real "ABC123").

Estas dos tareas son distintas y se resuelven con herramientas distintas, y en nuestro sistema se usan **una después de la otra**: primero se encuentra la placa dentro de la foto (detección), y luego se lee el texto de esa placa ya recortada (OCR).

4.3 YOLO: el modelo usado para detectar la placa

Para la tarea de detección usamos una familia de modelos llamada **YOLO** (siglas de *You Only Look Once*, "solo miras una vez"), específicamente una versión llamada YOLO11, a través de la librería Ultralytics. La idea de YOLO, explicada de forma sencilla, es que el modelo mira la imagen completa una sola vez y, en ese mismo paso, dice qué objetos encontró y en qué posición exacta (representada como un rectángulo, llamado *caja delimitadora*) están ubicados. Es un modelo basado en **redes neuronales convolucionales**, un tipo de red diseñada específicamente para "entender" imágenes, procesándolas por capas que van reconociendo patrones cada vez más complejos: primero bordes y colores simples, luego formas, y finalmente objetos completos como una placa.

Un modelo YOLO no nace sabiendo encontrar placas: hay que **entrenarlo** mostrándole muchas imágenes en las que ya sabemos exactamente dónde está la placa (esa ubicación conocida se llama *etiqueta*). Durante el entrenamiento, el modelo compara su propia predicción contra la ubicación real, calcula qué tan lejos estuvo (a ese error se le llama **función de pérdida**), y ajusta sus parámetros internos para reducir ese error la próxima vez. Este proceso se repite muchísimas veces: cada pasada completa por todas las imágenes de entrenamiento se llama una **época**, y nuestro modelo se entrenó durante decenas de épocas (más detalle en la sección de metodología).

4.4 EasyOCR: el modelo usado para leer el texto

Una vez que sabemos dónde está la placa, necesitamos leer qué dice. Para eso usamos **EasyOCR**, una librería de reconocimiento óptico de caracteres también basada en redes neuronales, ya entrenada de antemano por sus creadores para reconocer letras y números en cualquier imagen. A EasyOCR se le puede restringir qué caracteres puede devolver (en nuestro caso, solo letras de la A a la Z y números del 0 al 9, porque eso es lo único que aparece en una placa), lo cual reduce la posibilidad de que "invente" símbolos que nunca aparecerían ahí.

4.5 Datos sintéticos: por qué "inventamos" las placas de entrenamiento

Entrenar un modelo como YOLO requiere miles de ejemplos etiquetados. Fotografiar y etiquetar a mano miles de placas reales habría tomado semanas o meses, un tiempo que no teníamos. Por eso usamos una técnica llamada **datos sintéticos**: en vez de fotografiar placas reales, las **generamos por código**, dibujando el rectángulo amarillo, el texto y el marco negro con un programa, y luego les aplicamos transformaciones que imitan cómo se ve una foto real (rotación, desenfoque, sombras, ruido, distintos fondos). Como el programa es el que dibuja la placa, sabe exactamente dónde quedó cada letra y cada borde, así que la etiqueta (la ubicación exacta) sale automáticamente, sin que nadie tenga que marcarla a

mano. Esta es una técnica real y usada en la industria de la inteligencia artificial, no un atajo exclusivo de un proyecto escolar.

Sin embargo, como se explica más adelante en los resultados, los datos sintéticos tienen un límite: un modelo entrenado *solo* con imágenes generadas por computador no necesariamente funciona igual de bien sobre fotografías reales, porque el mundo real tiene una variedad de fondos, texturas, luces y detalles físicos (como los tornillos que sostienen una placa) que un programa no siempre logra imitar. A esta diferencia se le conoce, en el campo de la inteligencia artificial, como la **brecha entre lo sintético y lo real** (*sim-to-real gap*), y es uno de los hallazgos centrales de este proyecto, medido y documentado con datos propios (ver secciones 7 y 8).

4.6 El formato de las placas colombianas

Nuestro sistema aprovecha una regla fija del mundo real para corregir errores: en Colombia, las placas de los vehículos siguen un formato exacto:

- **Carros:** 3 letras seguidas de 3 números (por ejemplo, ABC123).
- **Motocicletas:** 3 letras, 2 números y una letra final (por ejemplo, ABC12D).

Como el formato es conocido y fijo, cuando el sistema de lectura devuelve un texto que casi cuadra con uno de los dos formatos pero tiene un error típico de confusión (por ejemplo, un "0" leído como la letra "O", que se parecen mucho), se puede corregir automáticamente sin necesidad de entrenar nada nuevo: basta con aplicar la regla. Esta capa de corrección, construida con lógica de programación normal (no con inteligencia artificial), resultó ser, como se ve en los resultados, la mejora individual más grande de todo el sistema.

4.7 El papel de la inteligencia artificial junto con las demás herramientas, y el papel de los lenguajes de programación

Uno de los puntos que quisimos dejar claro en este proyecto es que **la inteligencia artificial no funciona sola**: es una pieza dentro de una cadena más larga de herramientas, y el lenguaje de programación es lo que conecta esa cadena.

Todo el sistema está escrito en **Python**, un lenguaje de programación elegido por tres razones prácticas: tiene una sintaxis relativamente sencilla de leer y escribir (importante porque somos estudiantes aprendiendo), cuenta con una comunidad enorme de personas que ya construyeron y publicaron gratuitamente las herramientas de inteligencia artificial que necesitábamos (así que no había que inventar el modelo de detección o el modelo de lectura desde cero), y es, hoy en día, el lenguaje más usado en el mundo específicamente para inteligencia artificial y ciencia de datos.

El papel de Python en este proyecto no es "hacer" la inteligencia artificial —eso lo hacen los modelos ya entrenados o en entrenamiento (YOLO, EasyOCR)—, sino **organizar y conectar** cada pieza de la cadena, como el director de una orquesta que no toca cada instrumento pero coordina cuándo entra cada uno:

1. **Ultralytics (YOLO):** recibe la imagen y entrega la ubicación (caja) donde probablemente está la placa. Esta pieza SÍ es inteligencia artificial: fue entrenada mostrándole miles de ejemplos.
2. **OpenCV:** una librería de procesamiento de imágenes (no de inteligencia artificial) que se usa para tareas más "mecánicas": recortar la región de la placa que encontró YOLO, convertirla a escala de grises, mejorar su contraste y ajustar su tamaño antes de pasarla al siguiente paso.
3. **EasyOCR:** recibe ese recorte ya preparado y entrega el texto leído. Esta pieza también es inteligencia artificial, pero ya viene entrenada de fábrica (no la entrenamos nosotros).
4. **Lógica de corrección (código Python normal, sin IA):** revisa si el texto que devolvió EasyOCR cuadra con el formato colombiano y corrige errores típicos de confusión entre letras y números parecidos.
5. **SQLite:** una base de datos liviana (un solo archivo, sin necesidad de un servidor aparte) donde se consulta si la placa ya corregida pertenece a un residente o a un visitante autorizado.
6. **Streamlit:** la herramienta que arma la interfaz visual (pantallas, botones, formularios) que ve el vigilante, el propietario o el administrador, sin tener que programar una página web desde cero.

Python es el que recibe la imagen al principio, la pasa de una herramienta a la siguiente en el orden correcto, y al final arma la respuesta que se muestra en pantalla. Ninguna de estas herramientas, por separado, resuelve el problema completo: la inteligencia artificial (YOLO y EasyOCR) aporta la parte de "ver" e "interpretar" la imagen, mientras que el resto del código aporta la parte de "aplicar reglas conocidas" y "conectar con la información del conjunto residencial" — dos tareas que, aunque el sistema completo se sienta "inteligente", no requieren inteligencia artificial en sí mismas, sino programación tradicional bien organizada.

4.8 El entrenamiento del detector, explicado paso a paso

Como este es un concepto central del proyecto, vale la pena explicar con más detalle qué significa "entrenar" el detector de placas, porque es distinto de simplemente "programarlo":

1. Se generan miles de imágenes sintéticas de placas, cada una con su ubicación exacta conocida (la etiqueta).

2. Ese conjunto de imágenes se divide en dos partes: la mayoría para *entrenamiento* (de las que el modelo aprende) y una porción más pequeña para *validación* (imágenes que el modelo nunca usa para aprender, solo para comprobar si lo que aprendió realmente sirve con datos que no ha visto).
3. El entrenamiento se ejecuta en Google Colab, un servicio gratuito que presta temporalmente una tarjeta gráfica (GPU). Una GPU hace muchísimas operaciones matemáticas en paralelo, lo cual es exactamente lo que necesita el entrenamiento de una red neuronal; sin ella, el mismo entrenamiento en un computador normal tomaría muchas horas en vez de minutos.
4. Durante cada época (cada pasada completa por las imágenes de entrenamiento), el modelo predice dónde cree que está la placa en cada imagen, compara su predicción con la ubicación real (la etiqueta), calcula el error (la función de pérdida) y ajusta sus parámetros internos para reducir ese error la próxima vez.
5. Al terminar el entrenamiento se obtiene un archivo (en nuestro caso, `best.pt`) que contiene todos los parámetros ya ajustados: ese archivo *es* el modelo entrenado, listo para usarse sin necesidad de volver a entrenar cada vez que se detecta una placa nueva.

Este proceso se repitió **varias veces** a lo largo del proyecto (lo llamamos "vueltas de entrenamiento"), porque cada vez que medíamos el resultado sobre fotos reales encontrábamos algo que mejorar en los datos de entrenamiento, lo corregíamos, y volvíamos a entrenar. Esa parte del trabajo se describe con detalle en las secciones 7 y 8.

4.9 Antecedentes

Los sistemas de reconocimiento automático de placas (conocidos internacionalmente por la sigla **ALPR**, *Automatic License Plate Recognition*) no son una idea nueva: se usan hace años en peajes electrónicos, parqueaderos automatizados y sistemas de seguridad, incluyendo algunos ya en funcionamiento en Colombia. La gran mayoría de esos sistemas son productos comerciales cerrados, construidos por empresas, con cámaras especializadas y un costo que los hace poco accesibles para un conjunto residencial pequeño o mediano.

Este proyecto no pretende competir con esos sistemas profesionales ni igualar su precisión. Su aporte es distinto: demostrar, con fines educativos y usando únicamente herramientas de acceso libre y equipos comunes, que es posible construir una versión funcional del mismo concepto, entendiendo y midiendo en el camino qué tan difícil es realmente el problema y por qué los sistemas comerciales invierten tanto en resolverlo bien.

---

5. Formulación de la hipótesis

A partir de la pregunta de investigación, formulamos dos hipótesis relacionadas, una principal y una secundaria que surgió de la teoría (sección 4.5) y que decidimos poner a prueba explícitamente:

Hipótesis principal (H1): Un sistema de reconocimiento de placas construido con un detector entrenado sobre imágenes sintéticas y un módulo de lectura con corrección basada en el formato colombiano puede alcanzar una precisión de lectura exacta igual o superior al 70% sobre fotografías reales de vehículos, y esa precisión disminuye de forma notoria cuando aumenta el ángulo de inclinación entre la cámara y la placa.

- **Variable independiente:** las condiciones de captura de la imagen (principalmente el ángulo de la cámara respecto a la placa; también la distancia, la iluminación y el tipo de vehículo).
- **Variable dependiente:** la precisión de lectura del sistema, medida como el porcentaje de placas leídas de forma exactamente correcta.

Hipótesis secundaria (H2): Un modelo de detección entrenado únicamente con imágenes sintéticas tendrá un desempeño notoriamente menor sobre fotografías reales que sobre imágenes sintéticas de prueba, debido a la diferencia entre las condiciones del entrenamiento y las condiciones del mundo real.

Ambas hipótesis son refutables: si al medir encontráramos que la precisión no cambia con el ángulo, o que el modelo funciona igual de bien en fotos reales que en imágenes sintéticas, H1 y H2 quedarían rechazadas. Como se muestra en las secciones 7 y 8, esto es exactamente el tipo de comprobación que hicimos, con resultados que **confirman H2 con mucha claridad** y que dejan a **H1 parcialmente confirmada**, con una diferencia importante entre lo que se logra sobre imágenes sintéticas y lo que se logra —hasta el momento, en un proyecto que sigue en desarrollo— sobre fotografías reales.

---

6. Diseño metodológico

6.1 Enfoque y tipo de estudio

Este proyecto combina dos componentes: uno de **desarrollo tecnológico** (construir el sistema completo, pieza por pieza) y uno de **investigación cuantitativa experimental** (medir, con números, el efecto de distintas condiciones sobre la precisión del sistema ya construido). El tipo de estudio es **descriptivo y experimental**: describimos cómo se comporta el sistema y, además, manipulamos variables de forma controlada (ángulo,

distancia, iluminación, tipo de vehículo, con y sin cada mejora del sistema) para medir su efecto.

6.2 Población y muestra

- **Para entrenar el detector:** no se usó una "muestra" en el sentido tradicional, sino un conjunto de datos sintéticos generado por código: miles de imágenes de placas colombianas (formato carro y formato moto) con degradaciones que imitan condiciones reales (rotación, desenfoque, sombras, ruido, distintos fondos).
- **Para las pruebas controladas (banco de pruebas):** placas sintéticas generadas específicamente para cada experimento, con un mínimo de **30 repeticiones por condición**, siguiendo el protocolo explicado más abajo.
- **Para las pruebas con datos reales:** una muestra de **19 fotografías reales** de vehículos (carros y una motocicleta), tomadas por los estudiantes en distintos municipios del área metropolitana de Medellín, más un video real de aproximadamente 28 segundos grabado en un parqueadero. Esta muestra, aunque pequeña, ya permitió identificar hallazgos importantes (ver sección 8); el proyecto contempla ampliarla en las próximas semanas.

6.3 Instrumentos de recolección

- **Cámara de teléfono celular**, para las fotografías y el video reales.
- **El sistema mismo**, que al procesar cada imagen genera automáticamente sus propios datos de precisión, tiempo de procesamiento y nivel de confianza — no se midió nada "a ojo".
- **Scripts de medición automatizada**, construidos específicamente para este proyecto, que ejecutan cada experimento y guardan los resultados en tablas (formato CSV) y gráficas, sin intervención manual una vez que arrancan.

6.4 Procedimiento (organizado por fases de trabajo)

El proyecto se organizó en fases semanales, cada una construyendo sobre la anterior:

1. **Estructura base y primer reconocimiento:** se armó la estructura del proyecto y una primera versión simple del sistema (detectar el vehículo completo, sin un modelo propio todavía, y leer lo que se pudiera dentro de esa región).
2. **Generador de placas sintéticas:** se construyó el programa que dibuja placas colombianas falsas y les aplica degradaciones realistas, y se hizo la primera medición de qué tan bien lee EasyOCR una placa sintética limpia.
3. **Entrenamiento del detector propio:** se armó un conjunto de datos en el formato que exige YOLO, y se entrenó (en Google Colab) un modelo propio dedicado exclusivamente a encontrar placas.

4. **Validador y corrección por posición:** se construyó la capa que corrige errores típicos de lectura aprovechando el formato conocido de las placas colombianas, y un mecanismo de "votación" para videos (acumular varias lecturas de los mismos fotogramas y quedarse con la más repetida).
5. **Base de datos y lógica de autorización:** se construyó la base de datos de apartamentos, residentes, vehículos y visitantes, y la lógica que decide si una placa reconocida está autorizada.
6. **Aplicación completa:** se construyó la interfaz visual con tres roles (vigilante, propietario, administración).
7. **Modo maqueta:** se agregaron ajustes pensados para cuando el sistema se pruebe sobre un carro a escala (región de interés de la cámara, indicador de calidad de imagen).
8. **Medición y experimentación formal:** se diseñó y ejecutó un banco de pruebas automatizado con siete experimentos distintos, cada uno con al menos 30 repeticiones por condición.
9. **Pruebas con datos reales y ajustes iterativos:** al recibir las primeras fotografías y video reales, se comparó el desempeño del sistema contra esos datos, se identificaron varias causas concretas de error, se corrigieron, y se volvió a medir — un ciclo que, como se explica en la sección 7, se repitió varias veces y sigue abierto.

6.5 Control de variables extrañas

Para que las comparaciones entre una versión del sistema y la siguiente fueran justas, se mantuvo fija la **misma muestra de 19 fotografías reales** a lo largo de todas las mediciones posteriores a su recolección: cada vez que se modificó una parte del sistema (el detector, el lector, el corrector), se volvió a medir sobre exactamente las mismas fotos, para poder atribuir cualquier cambio en el resultado a la modificación hecha y no a que se estuviera midiendo con datos distintos. De la misma manera, en los experimentos del banco de pruebas se cambió **una sola variable a la vez** (por ejemplo, solo el ángulo, dejando fijas la distancia, la iluminación y el desenfoque), siguiendo el principio básico del método experimental de aislar el efecto de cada factor.

6.6 Sobre la replicabilidad

Todo el procedimiento, incluyendo el código fuente completo, los parámetros exactos usados en cada experimento y los resultados en bruto, quedó registrado en el repositorio del proyecto (carpeta `resultados/` y archivo `BITACORA_EXPERIMENTOS.md`), de modo que cualquier persona con el mismo tipo de computador y las mismas herramientas gratuitas podría repetir cada experimento y comprobar los resultados aquí reportados.

---

7. Recolección de datos y experimentación

Toda la experimentación de este proyecto quedó registrada en una bitácora técnica detallada (`BITACORA_EXPERIMENTOS.md`), siguiendo el principio de que **nada se descarta por no cuadrar**: incluso los resultados inesperados o que al principio parecían un error se registraron y se investigaron, porque —como efectivamente pasó varias veces— terminaron siendo el hallazgo más valioso. A continuación se resume, en orden cronológico, qué se hizo y qué se registró en cada etapa. Los números completos y el análisis se presentan en la sección 8.

Primera medición (línea base). Se midió qué tan bien lee EasyOCR una placa sintética limpia, sin ninguna mejora todavía, sobre 100 placas de prueba. Se registró no solo el porcentaje de aciertos, sino también **cuáles caracteres se confunden más entre sí** (por ejemplo, el número "0" leído como la letra "O"), información que se usó después para construir la capa de corrección.

Entrenamiento del detector propio y primer contratiempo. Al entrenar por primera vez el modelo de detección en Google Colab, el proceso falló con un error técnico (una carpeta comprimida mal organizada). Se registró el error completo, se diagnosticó la causa exacta y se corrigió antes de continuar. Un segundo intento de entrenamiento resultó, sin que se notara al principio, estar corriendo sin la tarjeta gráfica activada (mucho más lento de lo esperado); también se registró y se corrigió.

Serie de mediciones para encontrar por qué "el recorte ajustado" no ayudaba. Se esperaba que usar el detector propio (que recorta solo la placa) mejorara la lectura frente a usar la imagen completa. La primera medición mostró lo contrario. En vez de descartar la idea, se investigó con varias hipótesis sucesivas (agregar margen al recorte, revisar la resolución de la imagen), cada una medida por separado, hasta encontrar la causa real (un problema de cómo se escalaba la imagen antes de leerla) y confirmar, con una nueva medición, que efectivamente el recorte sí ayudaba una vez corregido ese problema.

Medición del efecto de la corrección por formato colombiano, primero con un error metodológico (se le daba al corrector información que en la vida real no tendría disponible) que se detectó, se corrigió, y se volvió a medir con el comportamiento real del sistema.

Recepción de las primeras fotografías y video reales, y comparación sistemática del desempeño del sistema (que hasta ese momento solo se había probado con datos sintéticos) contra esos datos reales. Esto se repitió en varias rondas: cada vez que se identificaba una causa concreta de error (el modelo nunca había visto fondos reales como mallas o pavimento; el lector mezclaba el número de la placa con el nombre de la ciudad impreso debajo; un tornillo de la placa se confundía con la letra "I"), se corregía esa causa específica,

se volvía a entrenar o ajustar el sistema, y se repetía la medición sobre las mismas 19 fotografías, para poder comparar de forma justa.

Ejecución del banco de pruebas formal, con siete experimentos controlados (distancia, ángulo, iluminación, desenfoque de movimiento, carro contra moto, con y sin votación temporal en video, y tiempo de procesamiento por etapa), cada uno con 30 repeticiones por condición, tal como exige el protocolo de medición que seguimos (ver sección 6).

Todas las condiciones exactas de cada corrida (parámetros usados, fecha, qué versión del sistema se estaba probando) quedaron documentadas, y los datos crudos (no solo los promedios) se conservaron en la carpeta `resultados/` del proyecto, en formato de tabla (CSV), junto con las gráficas correspondientes.

---

8. Procesamiento y análisis de resultados

8.1 Precisión de lectura: evolución a lo largo del proyecto

La siguiente tabla resume cómo fue subiendo la precisión de lectura exacta (el porcentaje de placas leídas carácter por carácter sin ningún error) **sobre placas sintéticas de prueba**, a medida que se fueron agregando las distintas partes del sistema:

Etapa del sistema	Precisión exacta
Solo lectura de texto (OCR), sin ninguna mejora	74.0%
+ detector propio (una vez corregido el problema de resolución)	76.0%
+ corrección por formato colombiano	87.0%
+ corrección de un error puntual en esa corrección	**90.0%**

Análisis: el salto más grande, por lejos, no vino de la parte de inteligencia artificial (el detector o el lector), sino de la capa de **reglas del dominio** (el formato conocido de las placas colombianas): +11 puntos porcentuales de una sola mejora, casi sin costo de tiempo adicional. Esto tiene sentido a la luz de lo explicado en el marco teórico (sección 4.6): cuando existe una regla fija y confiable sobre cómo debe verse el resultado correcto, aprovecharla con lógica de programación normal puede ser más efectivo que pedirle más precisión a un modelo de inteligencia artificial.

8.2 La brecha entre lo sintético y lo real (confirmación de la hipótesis H2)

Esta es la comparación más importante del proyecto, porque pone a prueba directamente la hipótesis secundaria (H2):

Medición	Sobre imágenes sintéticas	Sobre fotografías reales
Confianza del detector de placas	99%	63.2% (primera versión)
Tasa de detección	—	63.2% → 100% → 89.5% (a lo largo de tres versiones del detector)
Lecturas exactas correctas	90.0%	0/19 → 3/19 → 4/19 → 5/19 → 7/19 (a lo largo de las correcciones sucesivas)

Análisis: la hipótesis H2 queda **confirmada con claridad**: el mismo modelo que acertaba 99% de las veces sobre imágenes sintéticas de validación caía a 63.2% sobre fotografías reales en su primera versión. Al investigar por qué, se encontró una causa concreta y medible: las imágenes sintéticas de entrenamiento siempre mostraban la placa ocupando la mayoría del encuadre (alrededor del 62% del ancho de la imagen), mientras que en una fotografía real tomada a distancia normal, la placa ocupa una fracción mucho más pequeña de la escena. El modelo había aprendido, sin que nadie se lo enseñara a propósito, que "una placa siempre es casi toda la imagen" — una regla falsa fuera del mundo sintético en el que se entrenó.

Corregir esto (haciendo que las imágenes de entrenamiento varíen la escala de la placa dentro del cuadro, agregando fondos reales extraídos de fotos y video, y agregando ejemplos de "aquí no hay ninguna placa") subió la tasa de detección real hasta el 100%, y luego se mantuvo alta (89.5%) al agregar más variedad de entrenamiento. Las lecturas exactas correctas sobre las mismas 19 fotografías subieron de manera sostenida con cada corrección: 0, luego 3, luego 4, luego 5, luego 7 de 19.

Este hallazgo coincide con lo que se explicó en el marco teórico sobre la brecha entre datos sintéticos y datos reales (sección 4.5): no es un fallo de programación aislado, sino una limitación conocida y esperada de entrenar modelos únicamente con datos generados por computador, y confirma con datos propios algo que la teoría ya anticipaba.

8.3 Resultados del banco de pruebas controlado (7 experimentos, 30 repeticiones cada uno)

Experimento	Resultado	Interpretación
Ángulo de la cámara	63% (0°) → 47% (15°) → 13% (30°) → 0% (45° o más)	Caída fuerte y consistente: confirma la parte de la hipótesis H1 sobre el ángulo. La cámara debe quedar centrada respecto a la trayectoria del vehículo.
Iluminación	Mejor en condiciones intermedias (76-80%), peor en muy oscuro o muy iluminado (57-63%)	Recomienda luz difusa, no directa, sobre la placa (que es reflectiva).
Desenfoque de movimiento	Estable hasta cierto punto (73-77%), cae fuerte después (20%, luego 7%)	Recomienda que el vehículo pase despacio por la zona de lectura.
Carro contra moto	Carro 70%, moto 60%	Las placas de moto (con una letra adicional al final) son algo más difíciles de leer bien.
Distancia (simulada por computador)	Sin tendencia clara	Resultado no concluyente: la forma en que se simuló la distancia (achicando y agrandando la imagen) no reproduce bien el desenfoque real de una cámara lejana. Pendiente de repetir con fotos reales tomadas a distintas distancias.
Votación temporal en video (varias lecturas seguidas, quedarse con la más repetida)	Una sola lectura 73.3% contra 70.0% con votación (peor)	Resultado contraintuitivo, también no concluyente: la simulación generó cada fotograma de forma independiente, sin el parecido natural que tienen los fotogramas consecutivos de un video real. Pendiente de repetir con un video real de un vehículo pasando.
Tiempo de procesamiento por etapa	Detección: 33 ms. Lectura de texto: entre 1.9 y 4.9 segundos según la versión. Consulta a la base de datos: menos de 1 milisegundo.	La lectura de texto (OCR) es, con mucha diferencia, la parte más lenta de todo el sistema — el "cuello de botella".

Análisis general: los experimentos de ángulo, iluminación, desenfoque y tipo de vehículo dieron resultados claros y coherentes con lo esperado, y confirman la parte de la hipótesis H1 relacionada con el ángulo de la cámara. Los experimentos de distancia y de votación temporal en video, en cambio, dieron resultados **no concluyentes**, y en vez de forzar una interpretación se documentó honestamente por qué (la forma sintética de simular esas dos

condiciones no logra imitar bien el fenómeno físico real). Esto es consistente con el criterio de que un resultado inesperado no se descarta, sino que se investiga y se reporta con honestidad — incluyendo cuándo la causa probable es una limitación del propio método de medición, no del sistema que se está evaluando.

8.4 Un hallazgo adicional: no toda mejora beneficia todos los casos por igual

Al comparar dos versiones sucesivas del detector sobre las mismas 19 fotografías reales, se encontró que una fotografía que antes se leía perfectamente (con alta confianza) dejó de detectarse después de una nueva versión del modelo, aunque el resultado general mejoró en la mayoría de los demás casos. Este resultado, aunque pequeño, se registró con honestidad en la bitácora del proyecto: sirve como evidencia de que reentrenar un modelo con datos distintos no garantiza una mejora pareja en todos los casos, un comportamiento normal y documentado en el campo del aprendizaje automático.

---

9. Conclusiones

9.1 Respuesta a la pregunta de investigación

Sí es posible construir, con herramientas de acceso libre y un computador común, un sistema de visión artificial que reconozca automáticamente placas vehiculares colombianas de carro y de moto. El sistema construido en este proyecto alcanza una precisión de lectura exacta del **90% sobre placas sintéticas de prueba**, y —en la fase actual de desarrollo— del **7 de cada 19 (aproximadamente 37%) sobre fotografías reales**, con una tendencia claramente ascendente a medida que se identifican y corrigen causas concretas de error. Entre los factores medidos, el que más afecta la precisión es el **ángulo de la cámara respecto a la placa**, seguido por el **desenfoque de movimiento** y la **iluminación extrema**; y, de forma más importante aún, la **diferencia entre datos de entrenamiento sintéticos y condiciones reales** resultó ser, con diferencia, el factor de mayor impacto en todo el proyecto.

9.2 Sobre las hipótesis

- **Hipótesis secundaria (H2): confirmada con claridad.** El desempeño del sistema sobre datos sintéticos fue consistentemente mucho mayor que sobre fotografías reales, y se identificó y corrigió una causa concreta y medible de esa

diferencia (la falta de variedad de escala y de fondos reales en el entrenamiento), lo cual permitió reducir la brecha de forma sostenida a lo largo de varias correcciones.

- **Hipótesis principal (H1): aceptada parcialmente.** La parte de H1 sobre el efecto del ángulo de la cámara quedó confirmada con datos claros (la precisión cae de forma fuerte y consistente al aumentar el ángulo). La parte referida al umbral de 70% de precisión sobre fotografías reales **todavía no se alcanza** en la fase actual del proyecto (el resultado real más reciente es de aproximadamente 37%, frente al 90% ya logrado sobre datos sintéticos), aunque el camino recorrido —de 0% a 37% en varias correcciones sucesivas, cada una con una causa identificada y corregida— sugiere que ese umbral es alcanzable si el proyecto continúa su desarrollo. Por eso, y no por falta de rigor, esta hipótesis no se declara ni aceptada ni rechazada de forma definitiva, sino **parcialmente confirmada, con desarrollo en curso**.

9.3 Limitaciones del estudio

- La muestra de fotografías reales (19) es todavía pequeña, lo que hace que cada fotografía individual tenga un peso alto en el porcentaje final (cada una equivale a más de 5 puntos porcentuales); los resultados sobre datos reales deben leerse con esa cautela hasta ampliar la muestra.
- Dos de los siete experimentos del banco de pruebas (distancia y votación temporal en video) usaron simulaciones por computador que no lograron imitar bien el fenómeno físico real, así que sus resultados no se consideran concluyentes todavía.
- El proyecto no se ha probado aún sobre la maqueta física a escala planeada, ni sobre una cámara fija instalada en las condiciones reales de una portería; todas las pruebas reales hasta el momento provienen de fotografías tomadas con teléfono celular en condiciones variadas de la calle.
- Como se explicó desde el inicio del documento, **el proyecto sigue en fase de construcción**: al momento de escribir estas conclusiones hay una nueva versión del sistema ya preparada (que incorpora fotografías reales directamente como ejemplos de entrenamiento) pendiente de entrenar y medir.

9.4 Recomendaciones y trabajo futuro

- Completar el ciclo de entrenamiento pendiente (incorporando fotografías reales ya identificadas como ejemplos de entrenamiento directos, no solo como fondo) y volver a medir sobre la misma muestra de 19 fotografías para mantener la comparación justa.
- Ampliar la muestra de fotografías y video reales, priorizando **variedad de entornos** (distintos lugares, horas del día e iluminación) sobre simplemente más cantidad del mismo entorno, tal como sugiere la teoría sobre datos sintéticos y la brecha con el mundo real.

- Repetir los experimentos de distancia y votación temporal con datos físicos reales (fotografías a distancias controladas y video real de un vehículo en movimiento), en vez de las simulaciones usadas hasta ahora.
- Construir y probar la maqueta física a escala, para medir el sistema en condiciones más parecidas a las de una portería real, incluyendo la recomendación —ya derivada de los propios experimentos— de ubicar la cámara centrada respecto al carril del vehículo y con iluminación difusa, no directa.
- Evaluar si una señal adicional (por ejemplo, la clasificación de si el vehículo es carro o moto, tomada de un detector de vehículos general) permitiría cerrar la diferencia que todavía existe entre el desempeño del corrector cuando conoce el tipo de placa de antemano y cuando debe adivinarlo.

Una hipótesis que no se confirma del todo, como ocurre aquí con H1, **no es un resultado fallido**: es información real y valiosa sobre dónde está, en este momento, el límite del sistema, y hacia dónde debe dirigirse el trabajo que sigue.

Este documento resume el trabajo realizado hasta la fecha indicada en la portada. Todos los datos, tablas y gráficas mencionados provienen de mediciones reales del sistema, registradas en la carpeta `resultados/` *y en el archivo* `BITACORA_EXPERIMENTOS.md` *del proyecto, disponibles para consulta y verificación.*